



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Yazılım Geliştirme I

Sınıflar: Kapsülleme ve Kitap Sınıfı

Hafta 3

Sınıflar: Kapsülleme ve Kitap Sınıfı

Doç. Dr. Tolga Berber

Bilgisayar Bilimleri · <https://avesis.ktu.edu.tr/tberber>

1 • Ön-öğretim Neden Önemli?

C tarzı bir `struct Kitap { std::string baslik; int yıl; };` tasarımında, programınızın herhangi bir köşesinden gelen bir satır `k.yıl = -1500;` yazabilir — hiçbir şey buna engel olmaz.

- Proje büyüyüp onlarca fonksiyon aynı yapıya dokundukça, “kim, ne zaman, hangi alanı geçersiz bir değere düşürdü?” sorusunun izini sürmek neredeyse imkânsızlaşır
- Bu hafta veriyi kimin nasıl değiştirebileceğine artık serbest fonksiyonların değil, **tipin kendisinin** karar vermesini sağlayacaksınız

Bu Hafta

🎯 Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

- 1 Bir `struct`+serbest-fonksiyon tasarımının veri bütünlüğünü neden koruyamadığını açıklamak
- 2 `public/private` erişim belirleyicileriyle kapsülleme uygulamak
- 3 Kurucu, üye başlatma listesi, yıkıcı yazmak; `this`'in ne işe yaradığını açıklamak
- 4 Kitap sınıfını `private` alanlar + `public` arayüzle tasarlamak
- 5 Bir sınıfı `.hpp/.cpp` olarak ayırıp namespace içinde kurmak

BÖLÜM 1

Ünite 1 — Neden Sınıf?

Neden Sınıf Kurucu-Yıkıcı Kitap Sınıfı Kütüphane Özet

2. İçerik Struct + Serbest Fonksiyon: Nerede Kırılır?

Geçen hafta bellek ve fonksiyonlarla çalıştınız; bu hafta onları veriyle aynı çatı altında topluyorsunuz. Önce mevcut C tarzı tasarıma bakın:

```
1 struct Kitap {  
2     std::string baslik;  
3     int yıl;  
4 }; // + bu veriyi okuyan/yazan serbest fonksiyonlar
```

Kod tabanının uzak bir köşesinde biri şunu yazabilir:

```
k.yıl = -3;
```

Derleyici buna hiç itiraz etmez

`yıl` bir `int` alandır, `-3` de geçerli bir değerdir — tip açısından kusursuz bir atamadır. `struct`'ın “yıl pozitif olmalı” diyen bir kuralı yoktur.

Sınıf: Veri + Davranış + Erişim Bir Arada

Tanım: Sınıf ve nesne

Bir *sınıf* (*class*), veriyi (alanlar) ve o veri üzerinde çalışan davranışı (üye fonksiyonlar) tek bir kullanıcı tipi altında bir arada tanımlar; ayrıca bu alanlara sınıf dışından kimin erişebileceğini de kendisi belirler.

- *Nesne* (*object*): bir sınıftan üretilmiş somut örnek
- Kural artık programcının hatırlamasına kalan bir uzlaşma değil, **sınıfın tanımına gömülü, derleyicinin zorla uyguladığı** bir yapı

Analoji: Otomat Makinesi

Analoji

Bir otomatın önünde durduğunuzda içindeki ürünlere, raflara ya da para kutusuna elle uzanamazsınız; makineyle ilişkiniz tamamen dıştaki birkaç düğme ve para yuvası üzerinden yürür. Stoğun ne kadar azaldığını, kasadaki paranın ne kadar arttığını makine kendi içinde, siz hiç görmeden yönetir. Camı kırıp içeri uzanmak — arayüzü atlayıp iç düzene doğrudan müdahale etmek — makinenin kurallarını çiğnemek olur.

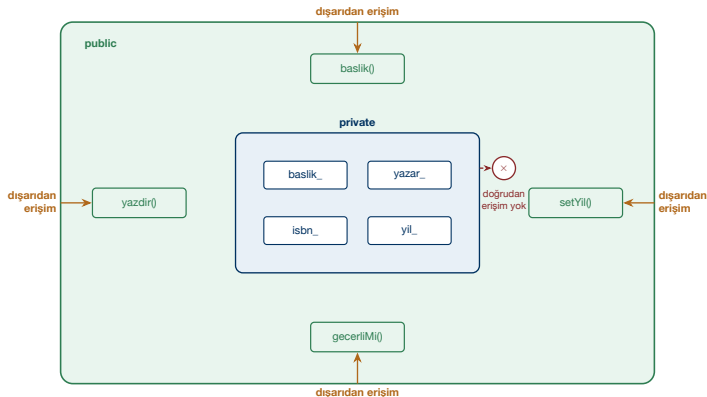
- Para yuvası + tuşlar → **public** arayüz
- Stok + kasa (iç durum) → **private** alanlar
- Camı kırmak → **private** alana dışarıdan doğrudan erişim

public / private: Erişim Belirleyiciler

```
1 struct KitapStruct { int yil; }; // varsayılan erişim: public
2
3 class KitapClass {
4 public:
5     void setYil(int yil) { if (yil > 0) yil_ = yil; } // kural burada
6 private:
7     int yil_; // varsayılan erişim: private
8 };
```

- **public**: sınıf dışından *herhangi* bir kod erişebilir
- **private**: yalnızca sınıfın kendi üye fonksiyonları erişebilir
- Tek fark: **struct** varsayılan **public**, **class** varsayılan **private**

Kapsülleme: Çekirdek ve Kabuk



private veri çekirdeği, yalnızca public arayüz kabuğu üzerinden erişilebilir olur.

BÖLÜM 2

Ünite 2 — Kurucu ve Yıkıcı

[Neden Sınıf](#) [Kurucu-Yıkıcı](#) [Kitap Sınıfı](#) [Kütüphane](#) [Özet](#)

2 • İçerik Kurucu: Nesnenin İlk Anı

Tanım: Kurucu

Bir *kurucu* (*constructor*), sınıfla aynı adı taşıyan, dönüş tipi olmayan özel bir üye fonksiyondur; bir nesne oluşturulduğu anda derleyici tarafından *otomatik* çağrılır.

- `Kitap k("Nutuk", "Mustafa Kemal", "9789751000000", 1927);` → kurucu otomatik çalışır, siz elle çağırmazsınız
- Nesnenin var olması ile doldurulması **aynı, bölünemez adımdır**
- Sınıfınızda kurucu yoksa derleyicinin ürettiği varsayılan kurucu alanları *ilklemez* — temel tipler çöp değerde kalır

Üye Başlatma Listesi mi, Gövdede Atama mı?

```
1 // Govdede atama: uye once (varsayılan) ilklenir, sonra üzerine  
   yazılır  
2 Kitap::Kitap(std::string b, int y) { baslik_ = b; yıl_ = y; }  
3  
4 // Uye baslatma listesi: doğrudan ilklenir (tercih edilen)  
5 Kitap::Kitap(std::string b, int y) : baslik_(b), yıl_(y) {}
```

- Gövdede atama: **iki** işlem — önce kur, sonra üzerine yaz
- Üye başlatma listesi: **tek** işlem — doğrudan ilkle
- `std::string` gibi üyelelerde fark somut bir maliyet farkıdır

const / Referans Üyeler İçin Zorunluluk

```
1 class Ornek {  
2     const int sabit_;  
3 public:  
4     Ornek(int deger) : sabit_(deger) {} // ZORUNLU: yalnız böyle  
        ilklenir  
5     // Ornek(int deger) { sabit_ = deger; } // DERLEME HATASI  
6 };
```

Neden zorunlu?

const bir üye bir kez ilklendikten sonra bir daha *hiçbir zaman* değiştirilemez; gövdede atama bir “yeniden atama” sayılır. Aynı kısıt referans üyeler için de geçerlidir.

this İşaretçisi

```
1 void setYil(int yil) {  
2     this->yil_ = yil; // sol: uyenin kendisi, sag: gelen parametre  
3 }
```

- `this`: üye fonksiyonun çağrıldığı nesnenin adresini tutan örtük işaretçi
- `a.yazdir()` çağrısında `this = &a`; `b.yazdir()` çağrısında `this = &b`
- Parametre adı üye adıyla çakıştığında çakışmayı çözer

Yıkıcı: ~Kitap()

```
1 void islemYap() {  
2     Kitap k("Nutuk", "M. Kemal", "9789751000000", 1927);  
3     k.yazdir();  
4 } // <- kapsam sonu: yikici burada otomatik calisir
```

- Sınıf adının başına ~ eklenerek yazılır, parametre almaz
- Nesnenin ömrü bitince (kapsam sonu) derleyici **otomatik** çağırır
- **Kitap** bu hafta yalnızca `std::string/int` üyeler taşıyor — kaynak yönetimi Hafta 4'te derinleştirecek

BÖLÜM 3

Ünite 3 — Kitap Sınıfını İnşa Etmek

Neden Sınıf Kurucu-Yıkıcı **Kitap Sınıfı** Kütüphane Özet

2 • İçerik Private Alanlar: Sınıfın Verisi

```
16 std::string baslik_  
17 std::string yazar_  
18 std::string isbn_  
19 int yil_;
```

- Dört alan, hepsi `private` — sınıf dışından hiçbir kod doğrudan erişemez
- Sondaki `_`: üyeyi aynı adlı parametreden ayıran adlandırma kuralı (`baslik_` vs `baslik`)

Public Arayüz: Sınıfın Davranışı

```
8  Kitap(std::string baslik, std::string yazar, std::string isbn, int
    yıl);
9  const std::string& baslik() const { return baslik_; }
10 void setYil(int yıl);
11 bool gecerliMi() const;
12 void yazdir() const;
```

- Kurucu, bir *getter* (`baslik()`), kontrollü bir *setter* (`setYil()`)
- `gecerliMi()` ve `yazdir()` — sınıfla dışarıdan kurabileceğiniz **tek** ilişki bu beş bildirimdir

const Üye Fonksiyonlar

```
1 class KitapOrnek {  
2     public:  
3         void yazdirDeneme() const {  
4             yıl_ = 1980; // DERLEME HATASI: const govde icinde uyeye  
                           yazilamaz  
5         }  
6     private:  
7         int yıl_;  
8 };
```

- `const` üye fonksiyonda `this`, `const Kitap*` olur
- `baslik()`, `gecerliMi()`, `yazdir()` yalnızca okur → `const`; kurucu ve `setYıl()` nesneyi değiştirir → `const` değil

Doğrulama Neden Sınıfın İçinde Yaşar?

```
11 bool Kitap::gecerliMi() const {  
12     return !baslik_.empty() && isbn_.size() == 13 && yil_ > 0;  
13 }
```

Neden üye fonksiyon, neden serbest fonksiyon değil?

Bu üç kural `Kitap`'ın **değişmezini** (invariantını) tanımlar — sınıfın kendi sorumluluğudur. Serbest bir fonksiyon olarak yazılsaydı, `baslik_/isbn_/yil_`'e erişemezdi (hepsi `private`).

Bir Kitap Nesnesi



Kurucu çağırıldıktan sonra alanların bellekte aldığı değerler.

BÖLÜM 4

Ünite 4 — Kütüphaneyi Kurmak

Neden Sınıf Kurucu-Yıkıcı Kitap Sınıfı **Kütüphane** Özet

2. İçerik .hpp/.cpp ve namespace kitaplık

```
1 #pragma once
2 #include <string>
3
4 namespace kitaplik {
5     class Kitap { /* alanlar ve arayuz */ };
6 } // namespace kitaplik
```

- `#pragma once`: bu dosyayı bir derleme biriminde yalnızca bir kez oku — çoklu `#include` koruması
- `namespace kitaplik`: `Kitap` adını isim çakışmasından korur; dışarıdan `kitaplik::Kitap`
- `Kitap`, `kitaplik`'e eklenen **ilk** sınıf — dönem boyu büyüyecek (DRY: her sınıf aynı kalıbı tekrar eder)

Derleyip Çalıştırmak

>_ Terminal

```
$ g++ -std=c++20 kitap_demo.cpp kitap.cpp -o kitap_demo  
$ ./kitap_demo
```

Beklenen çıktı:

 Beklenen Çıktı

Nutuk – Mustafa Kemal (1927), ISBN: 9789751000000

`kitap.cpp` kaynak listesinden çıkarılsaydı bağlayıcı “tanımsız sembol” (*undefined reference*) hatasıyla dururdu — bildirim (`.hpp`) yeterli değildir, gövdeler (`.cpp`) de derlenmelidir.

BÖLÜM 5

Özet

Neden Sınıf Kurucu-Yıkıcı Kitap Sınıfı Kütüphane **Özet**

Özet

Özet

- `struct`+fonksiyon veriyi korumasız bırakır; `class` veri+davranış+erişimi birleştirir
- Kapsülleme: `public/private` ile kontrollü erişim
- Kurucu tutarlı doğurur; üye başlatma listesi daha verimli ilkler
- `this` nesneyi gösterir; yıkıcı kapsam sonunda otomatik çalışır
- `Kitap`: private alanlar + public arayüz — `kitaplik`'in ilk sınıfı

5 • İzleme Aktarım — Başka Nerede?

“İç durumu gizle, yalnız kontrollü arayüzden eriştir” ilkesi yalnız **Kitap**’a özgü değildir:

- Kitaplık projesine dönem boyu eklenecek her sınıf (**Uye**, **Odunc**, **Raf**) aynı desenle iç durumunu koruyacak; doğrulama her tipte kendi kurallarını **kendi içinde** tutar
- Aynı fikir gündelik yaşamda da işler: bir otomat makinesinin para kutusuna elle uzanamazsınız, yalnız dıştaki tuş/para arayüzüyle etkileşirsiniz — makine kendi stok ve kasa durumunu kendisi yönetir
- **.hpp/.cpp** + **namespace** düzeni de ölçeklenen her C++ projesinin standart iskeletidir

Sırada Ne Var?

Bu hafta lab'da

Kitap sınıfını *kendi elinizle* yazacak, `.hpp/.cpp` olarak ayıracak ve derleyip çalıştıracaksınız.

- Ders notundaki **Kendinizi Sınavın** soruları her üniteyi kapsıyor: `private` gerekçesi, üye başlatma listesi vs atama, doğrulamanın sınıf içinde durma nedeni, `.hpp/.cpp` ayrımı
- Gelecek hafta: **Kitap** nesnelerinin sahip olabileceği kaynakları güvenli yönetmek — akıllı işaretçiler